Mémoriser ou généraliser ? Ce qu'apprennent les débruiteurs des modèles de diffusion

Travaux pratiques sur article

Master TRIED — Aymeric Chazottes

Contexte

L'énigme. Un modèle de diffusion apprend, à partir d'exemples, à produire de nouvelles images. Il le fait en apprenant un *débruiteur* : une fonction qui, à une image bruitée, associe une estimation de l'image propre. Or apprendre une loi de probabilité en grande dimension est réputé impossible sans hypothèses fortes : pour estimer sans a priori une densité d'images 80×80 codées sur 8 bits, il faudrait de l'ordre de 256^{6400} exemples. Les modèles de diffusion produisent pourtant des images de grande qualité à partir de quelques dizaines de milliers d'exemples. Deux explications s'opposent : soit ils *mémorisent* leurs exemples et les recombinent, soit ils *généralisent*, grâce à des a priori cachés dans l'architecture du réseau et dans l'apprentissage, ses *biais inductifs*.

L'article. Kadkhodaie, Guth, Simoncelli et Mallat [1] montrent que les deux régimes existent : un réseau entraîné sur peu d'images les mémorise, le même réseau entraîné sur 10^5 images généralise, au point que deux réseaux entraînés sur des images différentes apprennent presque la même fonction. Ils analysent ensuite ce que fait le débruiteur : il réduit les coefficients de l'image bruitée dans une base qui s'adapte à l'image, formée d'oscillations qui suivent ses contours (*base harmonique adaptée à la géométrie*, GAHB). Ce serait là le biais inductif qui permet de généraliser.

Le TP. On reprend ce raisonnement en le décomposant : ce que serait un débruiteur parfait (partie 2), ce que serait un débruiteur linéaire (partie 3), ce que fait le réseau selon la taille du jeu d'entraînement (partie 4), ce que son architecture lui impose (partie 5), et la base dans laquelle il travaille (parties 6 et 7).

Organisation

Le travail se fait dans les notebooks du dépôt forké, qui s'ouvrent dans Colab par leur bouton « Open in Colab ». Chaque notebook commence par une cellule d'installation à exécuter en premier : sur Colab, elle télécharge le code et les poids nécessaires ; en local, elle rend seulement le code importable. Les notebooks `TP_*` s'appuient sur le module `tp/tp_outils.py` (voir sa documentation par `help`), qui complète le code des auteurs sans le modifier. Tout ce qui a été ajouté au dépôt des auteurs est dans le dossier `tp/` ; les fichiers des auteurs ne sont pas modifiés, et leurs notebooks sont adaptés à Colab sous un autre nom (`tp/auteurs/<nom>_colab.ipynb`).

Notebook	Matériel	Durée CPU (GPU, non vérifié)	Parties
`TP_debruiteur_empirique`	CPU	2 min	2
`TP_gaussien_wiener`	CPU	2 min	3
`TP_entrainement_debruiteurs`	GPU ou CPU	28 min (25 min)	4
`Demo_UNet_CelebA80x80_colab` (auteurs)	GPU	— (15 min)	4
`TP_extraction_memorisation`	GPU ou CPU	14 min (10 min)	4
`TP_jacobien_base_adaptative`	CPU ou GPU	6 min (2 min)	5, 6
`Demo_UNet_Calpha80x80_colab` (auteurs)	GPU	— (5 min)	7

Les explications et les questions sont dans l'énoncé ; les notebooks exécutent les calculs et rappellent les formules qu'ils mettent en œuvre. Les réponses sont rédigées dans un compte rendu. Les questions marquées « (à la main) » ne demandent pas d'ordinateur.

Rappels

- *Loi normale* $\mathcal{N}(\mu, \Sigma)$ sur $\mathbb{R}^d$: densité proportionnelle à $\exp\left(-\frac{1}{2}(x-\mu)^\top \Sigma^{-1}(x-\mu)\right)$. Si $z \sim \mathcal{N}(0, I)$, chaque coordonnée $\langle z, e\rangle$ selon un vecteur unitaire e suit $\mathcal{N}(0,1)$, et $\mathbb{E}\|z\|^2 = d$.
- *Espérance conditionnelle* : parmi toutes les fonctions f, $\mathbb{E}\|x - f(y)\|^2$ est minimale pour $f(y) = \mathbb{E}[x \mid y]$.
- *Jacobien* d'une fonction $f : \mathbb{R}^d \to \mathbb{R}^d$: la matrice $\nabla f(y)$ de coefficients $\partial f_i / \partial y_j$; sa trace $\operatorname{tr} \nabla f(y) = \sum_i \partial f_i / \partial y_i$ est aussi la somme de ses valeurs propres.
- *PSNR* (rapport signal sur bruit de crête) d'une estimation $\hat{x}$ d'une image $x \in [0,1]^d$: $\text{PSNR} = 10 \log_{10}\left(1/\frac{1}{d}\|x - \hat{x}\|^2\right)$ en décibels. L'image bruitée $y = x + \sigma z$ a un PSNR d'environ $10\log_{10}(1/\sigma^2) = -20 \log_{10} \sigma$: c'est le « PSNR d'entrée ».
- *Similarité cosinus* de deux images u, v vues comme vecteurs : $\langle u, v\rangle / (\|u\|\|v\|)$. Dans le code des auteurs, on retire d'abord à chaque image sa moyenne (c'est alors un coefficient de corrélation).
- *Transformée de Fourier discrète* orthonormée d'une image $K \times K$: une isométrie ; ses vecteurs de base sont des sinusoïdes planes.

Graines et tirages aléatoires. Les tirages (bruits, échantillons, initialisations, choix d'images) ont leurs graines fixées dans les notebooks pour que les résultats soient reproductibles. Faites-les varier : un résultat n'a de valeur que s'il résiste à un changement de graine.

1 Débruiter, estimer le score, échantillonner

Objectifs

- Relier débruiteur optimal, score et densité ; lire ces relations dans le code des auteurs.
- Comprendre l'algorithme d'échantillonnage de l'article et le rôle de chacun de ses paramètres.
- Distinguer ce qui, dans tout cela, repose sur une hypothèse gaussienne.

Débruiter. On observe $y = x + z$, où x est une image propre de loi p (inconnue) et $z \sim \mathcal{N}(0, \sigma^2 I)$ un bruit blanc gaussien indépendant de x. Un débruiteur est une fonction f qui estime x à partir de y ; sa qualité se mesure par l'erreur quadratique moyenne

$$\text{MSE}(f, \sigma^2) = \mathbb{E}_{x,y}\|x - f(y)\|^2.$$

Le débruiteur optimal est l'espérance conditionnelle $f^\star(y) = \mathbb{E}[x \mid y]$. En pratique, un réseau f_θ est entraîné à minimiser l'erreur quadratique sur des couples (image propre, image bruitée) tirés d'un jeu d'entraînement.

La densité des images bruitées. La loi de y a pour densité $p_\sigma(y) = \int g_\sigma(y-x)\, p(x)\, dx$, où g_σ est la densité de $\mathcal{N}(0, \sigma^2 I)$: c'est p lissée par convolution avec une gaussienne. Quand σ croît, p_σ devient de plus en plus lisse, jusqu'à ressembler à une gaussienne très large ; la famille $(p_\sigma)_{\sigma \geq 0}$ est celle que parcourt un processus de diffusion.

Score et débruiteur (formule de Tweedie–Miyasawa). Le *score* de p_σ est le champ de vecteurs $\nabla_y \log p_\sigma(y)$. Il est lié au débruiteur optimal par

$$\nabla_y \log p_\sigma(y) = \frac{\mathbb{E}[x \mid y] - y}{\sigma^2}, \qquad \text{soit} \qquad f^\star(y) = y + \sigma^2 \nabla_y \log p_\sigma(y). \tag{1}$$

Un débruiteur appris fournit donc une estimation du score, $s_\theta(y) = (f_\theta(y) - y)/\sigma^2$.

Q1 — Démontrer la formule de Tweedie–Miyasawa (à la main)

(a) Calculez $\nabla_y g_\sigma(y-x)$, puis $\nabla_y p_\sigma(y)$ en dérivant sous l'intégrale. Déduisez-en (1) en utilisant la formule de Bayes $p(x \mid y) = g_\sigma(y-x)p(x)/p_\sigma(y)$.

(b) Quelle propriété du bruit avez-vous utilisée ? Pour un bruit de loi quelconque, la relation resterait-elle vraie ?

Débruiter à tous les niveaux de bruit, c'est apprendre la densité. L'article rappelle (d'après Song et al., 2021) que l'erreur du modèle génératif, mesurée par la divergence de Kullback–Leibler entre la loi des images p et la loi p_θ des échantillons produits, est contrôlée par l'écart entre l'erreur du débruiteur appris et celle du débruiteur optimal, intégré sur tous les niveaux de bruit :

$$D_{\mathrm{KL}}(p \,\|\, p_\theta) \le \int_0^\infty \big(\mathrm{MSE}(f_\theta, \sigma^2) - \mathrm{MSE}(f^\star, \sigma^2)\big)\, \sigma^{-3}\, d\sigma. \tag{2}$$

Bien débruiter à tous les niveaux suffit donc à bien modéliser la densité. Mais $f^\star$ est inconnu pour des photographies : on ne peut pas évaluer directement l'écart à l'optimum.

Q2 — Ce que dit et ne dit pas la borne (2) (à la main)

(a) En utilisant (1), montrez que l'erreur quadratique sur le score, $\mathbb{E}\|\nabla \log p_\sigma(y) - s_\theta(y)\|^2$, vaut $\sigma^{-4}\big(\mathrm{MSE}(f_\theta, \sigma^2) - \mathrm{MSE}(f^\star, \sigma^2)\big)$.

(b) Pourquoi le facteur σ^{-3} rend-il les faibles niveaux de bruit déterminants ? Qu'en déduire sur la façon de répartir les niveaux de bruit pendant l'entraînement ?

Un débruiteur aveugle. Les réseaux de l'article sont *universels et aveugles* : un seul réseau traite tous les niveaux de bruit, sans recevoir σ en entrée (contrairement aux réseaux des modèles de diffusion habituels, qui reçoivent le temps t, ou σ). Dans le code, le réseau ne renvoie pas l'image débruitée mais le *résidu* $r_\theta(y) = y - f_\theta(y)$, c'est-à-dire le bruit estimé : voir `trainer.py` (`target = noise`, `denoised = noisy - output`).

Q3 — Le score dans le code

Exprimez le score estimé $s_\theta(y)$ à l'aide du résidu $r_\theta(y)$ renvoyé par le réseau. Le réseau étant aveugle, comment l'algorithme d'échantillonnage peut-il connaître le niveau de bruit σ de l'image qu'il traite ? Repérez la ligne correspondante dans `univ_inv_sol` (`code/algorithm_inv_prob.py`).

Échantillonner. L'algorithme de l'article (Kadkhodaie et Simoncelli, 2021) part d'une image de bruit et monte le gradient de $\log p_\sigma$ tout en réinjectant un peu de bruit, avec un pas qui s'adapte de lui-même au niveau de bruit courant. En notant y_t l'image à l'étape t et $d = K^2$ la dimension :

Algorithme d'échantillonnage (version du code, `univ_inv_sol` pour la synthèse)

1. $y_0 = m + \sigma_0 z_0$, où m est l'image moyenne des données et $z_0 \sim \mathcal{N}(0, I)$;
2. pour $t = 1, 2, \ldots$ tant que $\sigma_t > \sigma_\infty$:
   - $r_t = r_\theta(y_{t-1})$ (résidu, $= -\sigma^2 \times$ score estimé) ;
   - $\sigma_t = \|r_t\|/\sqrt{d}$ (niveau de bruit effectif) ;
   - $h_t = h_0\, t/(1 + h_0(t-1))$ (pas croissant, de h_0 vers 1) ;
   - $\gamma_t = \sigma_t \sqrt{(1-\beta h_t)^2 - (1-h_t)^2}$;
   - $y_t = y_{t-1} - h_t\, r_t + \gamma_t\, z_t$, avec $z_t \sim \mathcal{N}(0, I)$;
3. renvoyer $y_t - r_\theta(y_t)$ (un dernier débruitage complet).

Réglages de l'article : $h_0 = 0{,}01$, $\beta = 0{,}1$, $\sigma_0 = 1$, $\sigma_\infty = 0{,}05$.

Q4 — Pourquoi cette valeur de γ_t ? (à la main)

On suppose que $y_{t-1} = x' + \sigma_t\, n$, où x' est une image plausible et n un bruit blanc de variance 1, et que le débruiteur est parfait : $r_t = \sigma_t\, n$.

(a) Montrez que $y_t = x' + (1 - h_t)\sigma_t\, n + \gamma_t z_t$ et calculez la variance par pixel du bruit de y_t. En déduire que $\sigma_{t+1} = (1 - \beta h_t)\,\sigma_t$.

(b) Que se passe-t-il pour $\beta = 1$? Pour β proche de 0 ? Quel est l'intérêt de réinjecter du bruit ?

(c) Estimez le nombre d'itérations nécessaires pour passer de $\sigma_0 = 1$ à $\sigma_\infty = 0{,}05$ avec les réglages de l'article (utilisez $\ln(1 - u) \approx -u$ et approchez la somme des h_t par une intégrale).

Ce que « gaussien » veut dire ici. Le mot revient partout ; il recouvre des hypothèses très différentes, qu'il faut distinguer.

Q5 — Qu'est-ce qui est gaussien ? (à la main)

Pour chacun des objets suivants, dites s'il est gaussien par hypothèse, par construction, approximativement, ou pas du tout, et ce que cette propriété permet : (a) le bruit z ajouté aux images ; (b) la loi des images propres p ; (c) la loi bruitée p_σ pour σ très grand ; (d) la loi a posteriori $p(x \mid y)$; (e) le bruit z_t réinjecté par l'échantillonneur ; (f) le résidu $r_\theta(y)$ d'un bon débruiteur.

2 Un débruiteur parfait ne généralise pas

Objectifs

- Calculer exactement le débruiteur optimal quand la loi est celle d'un jeu d'entraînement fini.
- Constater qu'il ne fait que recopier les images d'entraînement, et en tirer la question centrale de l'article.

La situation. Un réseau est entraîné à minimiser l'erreur de débruitage sur un jeu fini $\{x_1, \dots, x_N\}$. Il ne voit jamais la vraie loi p des images, seulement la *loi empirique* uniforme sur ces N images. Que ferait un réseau assez puissant pour atteindre le minimum exact de cette erreur ? On le calcule, puis on l'expérimente dans le notebook `TP_debruiteur_empirique`, sur des images synthétiques $\mathbf{C}^\alpha$ (décrites en partie 7) de taille 80×80, $d = 6\,400$.

Q6 — Le débruiteur optimal d'un jeu fini (à la main)

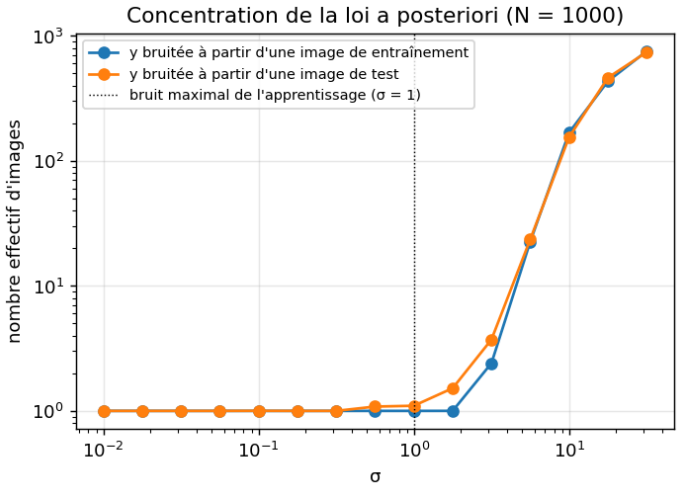
On suppose x uniforme sur $\{x_1, \dots, x_N\}$ et $y = x + \sigma z$.

(a) Montrez que la loi a posteriori est portée par les x_i, avec des poids $w_i(y) = P(x = x_i \mid y) \propto \exp\big(-\|y - x_i\|^2/2\sigma^2\big)$; en déduire $f^\star(y) = \sum_i w_i(y)\, x_i$. Reconnaissez une opération bien connue des réseaux de neurones.

(b) Que devient $f^\star(y)$ quand $\sigma \to 0$? Quand $\sigma \to \infty$? Que vaut alors l'erreur de $f^\star$ sur une image d'entraînement bruitée faiblement ? sur une image de test ?

(c) Montrez que le score correspondant pousse y vers une moyenne pondérée des x_i. Quels échantillons l'algorithme de la partie 1 produira-t-il avec ce score ?

Concentration de la loi a posteriori. Le notebook mesure la concentration des poids par le nombre effectif d'images $n_{\text{eff}} = \exp\big(-\sum_i w_i \log w_i\big)$ (1 si un seul poids est non nul, N si les poids sont égaux), en fonction de σ, pour $N = 1\,000$.

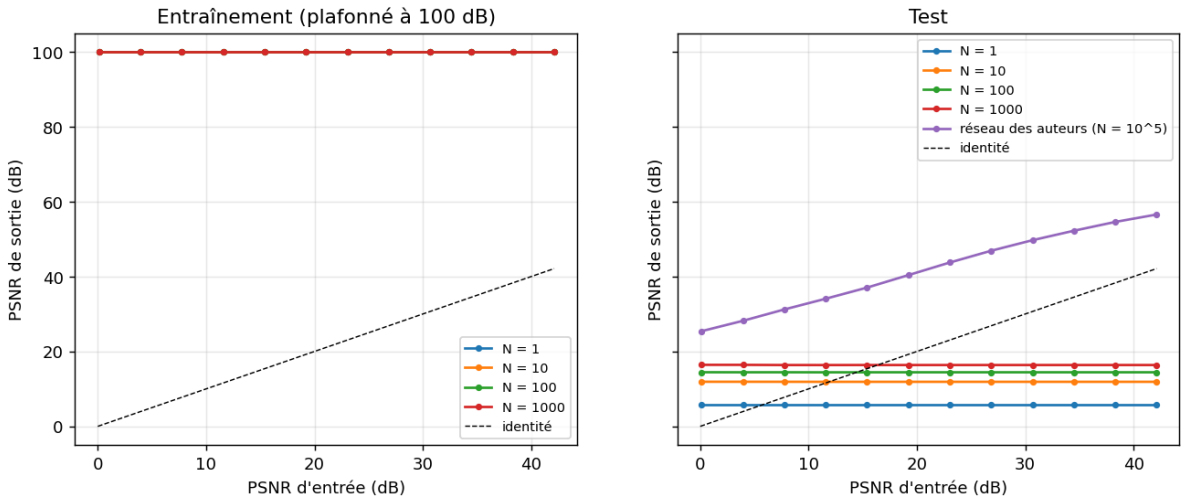


Q7 — Pourquoi la loi a posteriori est-elle concentrée ?

(a) Jusqu'à quel niveau de bruit la loi a posteriori reste-t-elle concentrée sur une seule image, pour une image d'entraînement et pour une image de test ? Comparez au niveau de bruit maximal utilisé pour entraîner les réseaux de l'article ($\sigma = 1$).

(b) Le notebook affiche la distance médiane d'une image à sa plus proche voisine dans le jeu d'entraînement (12,3). En comparant $\|y-x_i\|^2 - \|y-x_j\|^2$ à $2\sigma^2$, expliquez pourquoi les poids restent concentrés bien au-delà de $\sigma = 0{,}15$, l'écart par pixel entre deux images voisines.

Débruitage sur l'entraînement et sur le test. Comme dans la figure 1 de l'article, le notebook trace le PSNR de sortie en fonction du PSNR d'entrée, sur des images d'entraînement et de test, pour $N = 1, 10, 100, 1\,000$ (le PSNR d'une restitution parfaite, infini, est plafonné à 100 dB). La courbe de test du réseau des auteurs, entraîné sur 10^5 images $\mathbf{C}^4$, est ajoutée pour comparaison.

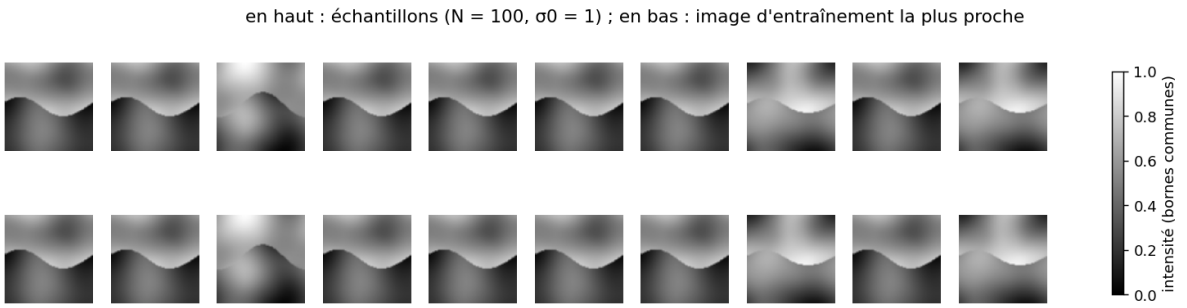


Q8 — Lire les courbes

(a) Expliquez la forme des courbes d'entraînement et de test du débruiteur optimal empirique. Comment le PSNR de test évolue-t-il avec N ? Que faudrait-il pour qu'il rejoigne la courbe du réseau ?

(b) Le réseau des auteurs a été entraîné sur 10^5 images, donc sur une loi empirique. S'il avait atteint le minimum exact de son erreur d'entraînement, quelle serait sa courbe de test ? Qu'en concluez-vous ?

Échantillonner avec le débruiteur parfait. L'échantillonneur de la partie 1 demande un débruiteur aveugle : `T.ResiduEmpiriqueAveugle` estime σ^2 par $\min_i \|y - x_i\|^2/d$. Le notebook produit 32 échantillons pour $N = 10$ et $N = 100$, avec $\sigma_0 = 1$ (réglage de l'article) puis $\sigma_0 = 10$, cherche pour chacun l'image d'entraînement la plus proche et compte les images distinctes obtenues.



Q9 — Des copies, mais pas toutes

(a) Quelle est la similarité des échantillons à leur plus proche voisin d'entraînement ?

(b) Avec $\sigma_0 = 1$, seules 2 des 10 images (et 6 des 100) sont produites ; avec $\sigma_0 = 10$, 10 sur 10 (et 25 sur 100). Expliquez ce phénomène à l'aide de la question 7. Les échantillons suivent-ils la loi empirique ?

Le jacobien du débruiteur parfait. Le jacobien d'un débruiteur décrit comment sa sortie réagit à une petite modification de l'entrée ; il est au cœur de la seconde moitié de l'article. Pour la loi empirique, il se calcule exactement.

Q10 — Jacobien du débruiteur empirique (à la main, puis notebook)

(a) Montrez que $\nabla w_i(y) = w_i(y)\,(x_i - f^\star(y))/\sigma^2$, puis que $\nabla f^\star(y) = \frac{1}{\sigma^2}\sum_i w_i(y)\,(x_i - f^\star(y))(x_i - f^\star(y))^\top = \mathrm{Cov}[x \mid y]/\sigma^2$. Quel est son rang au plus ? Que vaut-il quand un seul poids est non nul ?

(b) Le notebook se place à mi-chemin des deux images d'entraînement les plus proches (poids 0,73 et 0,27), compare la formule à la différentiation automatique (écart $4 \cdot 10^{-8}$) et trouve une seule valeur propre non nulle, 190, de vecteur propre $(x_i - x_j)/\|x_i - x_j\|$. Justifiez ces deux résultats. Une valeur propre de 190 vous surprend-elle ?

(c) L'erreur optimale vérifie $\mathrm{MSE}(f^\star, \sigma^2) = \sigma^2\,\mathbb{E}_y[\mathrm{tr}\,\nabla f^\star(y)]$ (on la démontrera en partie 6). Le notebook le vérifie par Monte-Carlo pour $N = 100$: les deux membres sont nuls jusqu'à $\sigma = 1$, puis valent 4,1 et 4,1 à $\sigma = 3$, 243 et 245 à $\sigma = 10$. Pourquoi sont-ils nuls pour $\sigma \le 1$?

3 Le cas gaussien : débruiter dans une base fixe

Objectifs

- Établir que, pour une loi gaussienne, le débruiteur optimal est linéaire et réduit les coefficients de l'image dans une base fixe ; pour une loi stationnaire, cette base est celle de Fourier.
- Relier la vitesse de décroissance des coefficients dans une base à la pente des courbes PSNR.
- Mesurer ce qu'un débruiteur linéaire perd sur des images qui ne sont pas gaussiennes.

Le filtre de Wiener. Si $x \sim \mathcal{N}(\mu, \Sigma)$, le couple (x, y) est gaussien et la loi a posteriori aussi : le débruiteur optimal est

$$f^\star(y) = \mu + \Sigma(\Sigma + \sigma^2 I)^{-1}(y - \mu), \qquad \nabla f^\star(y) = \Sigma(\Sigma + \sigma^2 I)^{-1}. \tag{3}$$

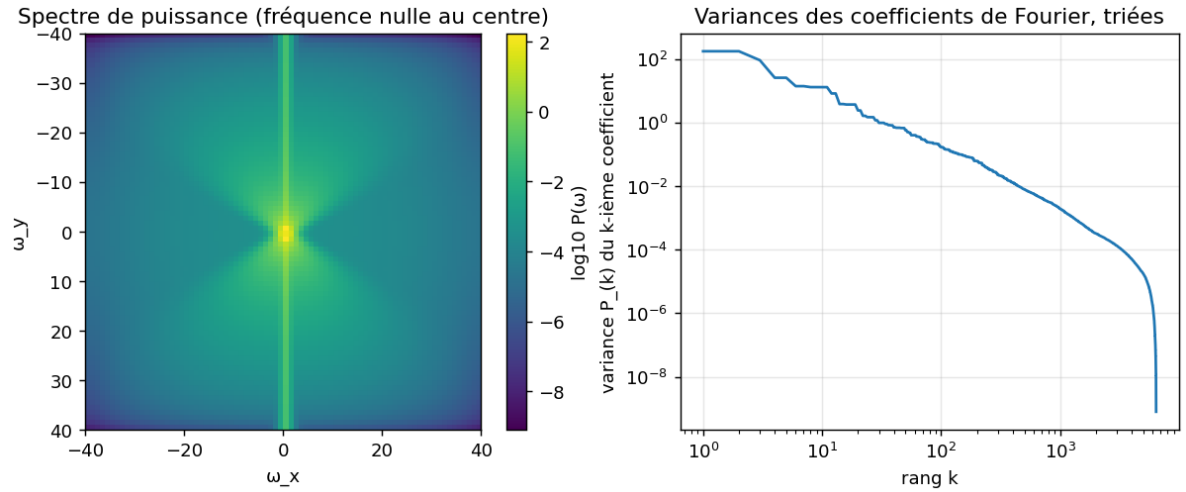
Q11 — Le filtre de Wiener (à la main)

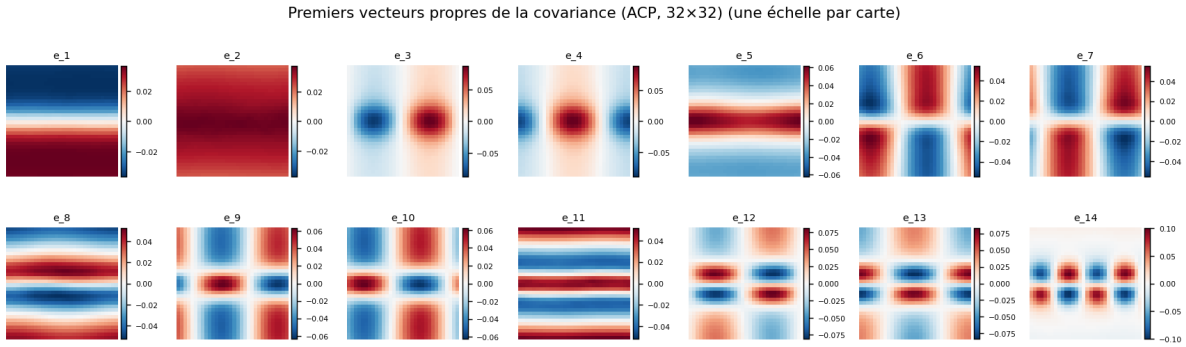
(a) On diagonalise $\Sigma = \sum_k s_k\, e_k e_k^\top$ (ACP). Montrez que $f^\star$ multiplie le coefficient $\langle y - \mu, e_k\rangle$ par $\lambda_k = s_k/(s_k + \sigma^2)$. Dans quel intervalle sont les λ_k ? Le jacobien dépend-il de y ?

(b) Montrez que l'erreur vaut $\mathrm{MSE} = \sum_k \sigma^2 s_k/(s_k + \sigma^2) = \sigma^2 \operatorname{tr} \nabla f^\star$.

(c) Une loi est *stationnaire* si ses statistiques sont invariantes par translation : la covariance entre deux pixels ne dépend que de leur décalage. Avec des bords périodiques, Σ est alors circulante. Pourquoi ses vecteurs propres sont-ils les sinusoïdes de Fourier, quelle que soit la loi ? Comment s'appellent les valeurs propres correspondantes ?

Les images $\mathbf{C}^\alpha$ sont-elles stationnaires ? Le notebook `TP_gaussien_wiener` estime, sur 20 000 images $\mathbf{C}^4$, la moyenne et le spectre de puissance (à gauche) et trace les variances des coefficients de Fourier triées par ordre décroissant (à droite). Il calcule aussi l'ACP sur des images réduites à 32×32, sans supposer la stationnarité.





Q12 — Spectre et ACP

(a) Le spectre est-il isotrope ? Qu'est-ce qui, dans les images, explique sa forme ?

(b) Les vecteurs propres de l'ACP ressemblent-ils à des sinusoïdes ? Qu'est-ce qui empêche qu'ils soient exactement des sinusoïdes de Fourier ?

Décroissance des coefficients et pente des courbes PSNR. L'article (section 3.1) relie la performance d'un débruiteur qui travaille dans une base fixe (e_k) à la vitesse de décroissance des coefficients de l'image dans cette base. Le raisonnement passe par un *oracle* : un débruiteur qui connaîtrait l'image propre x pour choisir ses facteurs de rétrécissement, $\hat{x} = \sum_k \lambda_k(x)\langle y, e_k\rangle e_k$. Aucun débruiteur réel ne fait mieux, et des seuillages qui n'utilisent que y font presque aussi bien (Donoho et Johnstone, 1994).

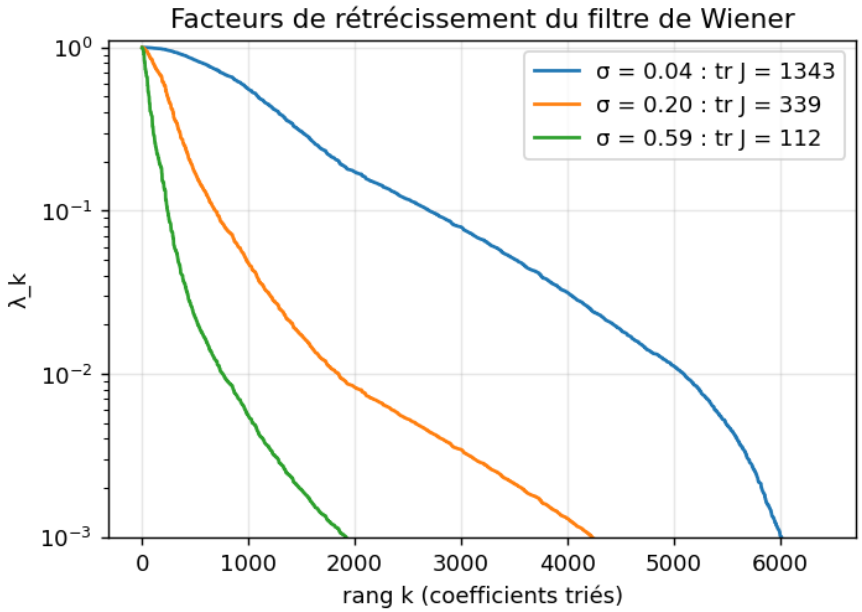
Q13 — L'oracle et la pente $\alpha/(\alpha+1)$ (à la main)

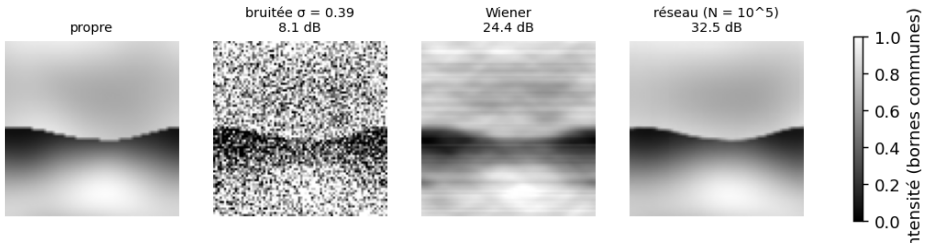
(a) Montrez que l'erreur de l'oracle vaut $\sum_k \big((1-\lambda_k)^2\langle x, e_k\rangle^2 + \lambda_k^2\sigma^2\big)$, minimale pour $\lambda_k = \langle x, e_k\rangle^2/(\langle x, e_k\rangle^2+\sigma^2)$, et qu'elle est alors comprise entre $\frac{1}{2}\sum_k \min(\langle x, e_k\rangle^2, \sigma^2)$ et $\sum_k \min(\langle x, e_k\rangle^2, \sigma^2)$.

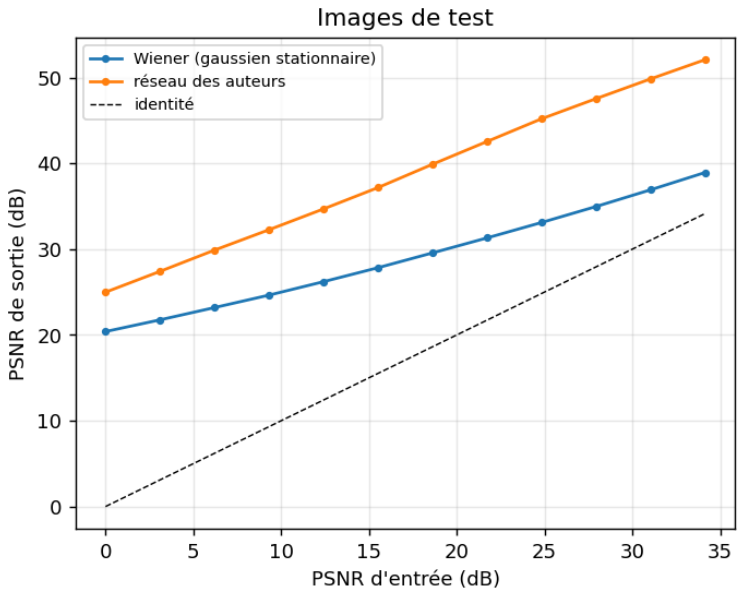
(b) On suppose que les coefficients, triés par ordre décroissant, vérifient $\langle x, e_{(k)}\rangle^2 \approx Ck^{-(\alpha+1)}$. Soit M le nombre de coefficients supérieurs à σ^2. Montrez que $\sum_k \min(\langle x, e_k\rangle^2, \sigma^2) = M\sigma^2 + \|x - x_M\|^2$ (où x_M est l'approximation de x par ses M plus grands coefficients) et que ces deux termes sont de l'ordre de $\sigma^{2\alpha/(\alpha+1)}$.

(c) En déduire que le PSNR de sortie croît asymptotiquement (petits bruits) comme $\frac{\alpha}{\alpha+1}$ fois le PSNR d'entrée. Que signifie une pente égale à 1 ? proche de 0 ?

Wiener face au réseau. Le notebook applique le filtre de Wiener (dans le domaine de Fourier, avec le spectre estimé) et le réseau des auteurs à 200 images de test.







Q14 — Pentes mesurées et pentes prédites

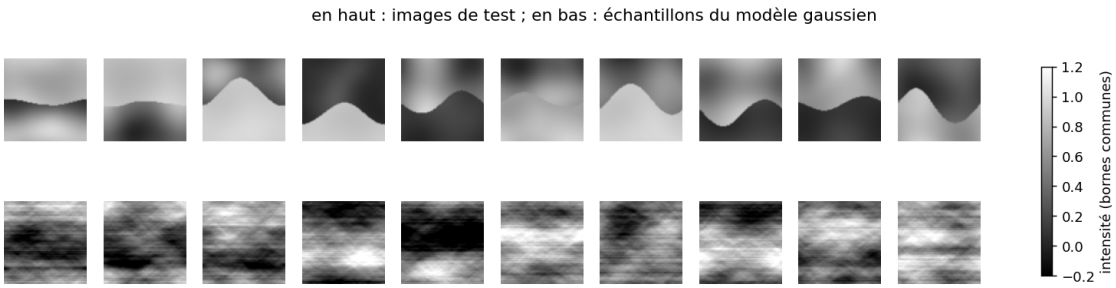
Le notebook mesure une pente de 0,59 pour Wiener et 0,80 pour le réseau (sur les PSNR d'entrée supérieurs à 15 dB). Il ajuste aussi la décroissance des variances de Fourier triées, $P_{(k)} \propto k^{-(\alpha_F+1)}$, sur les rangs où elles sont comparables aux σ^2 de l'ajustement, et trouve $\alpha_F = 1{,}26$.

(a) Quelle pente la question 13 prédit-elle pour Wiener ? Comparez à la mesure.

(b) Pour les images $\mathbf{C}^4$, la théorie (partie 7) donne une pente optimale de $\alpha/(\alpha+1) = 0{,}8$. Que conclure du réseau ? Où le filtre de Wiener perd-il le plus, d'après l'exemple débruité ?

(c) Le filtre de Wiener a une erreur environ 5 fois plus grande que le réseau à $\sigma = 100/255$. Est-ce parce que le spectre est mal estimé ?

Ce que « gaussien » veut dire : des échantillons. Le notebook tire des échantillons du modèle gaussien stationnaire, qui a exactement la même moyenne et le même spectre que les images.



Q15 — Même covariance, autres images

(a) Qu'est-ce qui distingue les échantillons gaussiens des images $\mathbf{C}^4$? Qu'est-ce que le modèle gaussien ne peut pas représenter ?

(b) Le notebook compare $\sigma^2 \operatorname{tr} \nabla f$ à l'erreur mesurée du filtre de Wiener, sur des échantillons gaussiens et sur les vraies images : 2,07 / 2,07 / 2,06 à $\sigma = 0{,}04$; 13,04 / 13,04 / 13,04 à $\sigma = 0{,}2$. Montrez que l'erreur d'un estimateur linéaire ne dépend que de la moyenne et de la covariance des données. Qu'en conclure sur ce que l'on gagne à sortir du linéaire ?

4 De la mémorisation à la généralisation

Objectifs

- Lire les deux expériences de l'article (section 2.2) : courbes de débruitage sur l'entraînement et le test, comparaison de deux modèles entraînés sur des données disjointes.
- Les reproduire à petite échelle en entraînant soi-même des débruiteurs, et repérer la mémorisation des modèles des auteurs, jusqu'à retrouver leurs images d'entraînement.

Les expériences de l'article. Les auteurs entraînent le même UNet sur des sous-ensembles de CelebA (80×80, niveaux de gris, doublons retirés) de taille $N = 1, 10, \ldots, 10^5$ (figure 1). Pour chaque taille, ils entraînent deux réseaux sur deux sous-ensembles *disjoints* S_1 et S_2, et génèrent un échantillon avec chacun à partir du même bruit initial (figure 2).

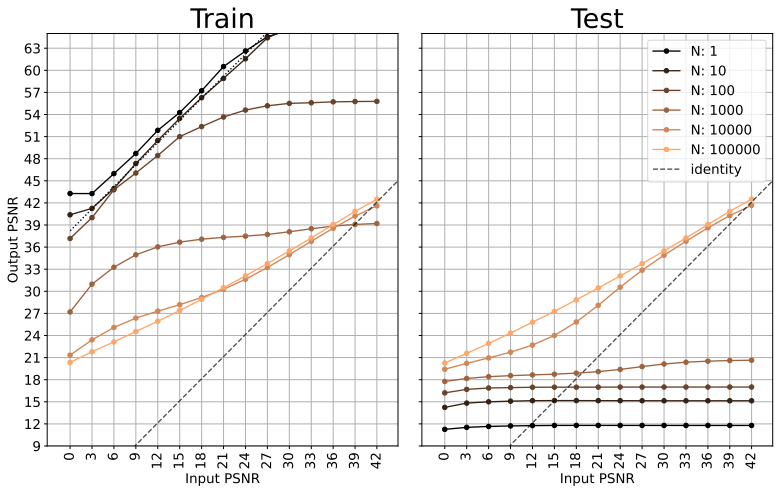


Figure 1 de l'article [1] : PSNR de sortie en fonction du PSNR d'entrée, sur l'entraînement (à gauche) et sur le test (à droite), pour $N = 1$ à 10^5.

Q16 — Lire la figure 1 de l'article

(a) Décrivez les courbes d'entraînement et de test pour $N = 1$ et $N = 10$. Rapprochez-les du débruiteur empirique de la partie 2. Pourquoi la courbe d'entraînement a-t-elle une pente 1 ?

(b) Que se passe-t-il pour $N = 1\,000$? pour $N = 10^5$? Pourquoi la performance sur l'entraînement *diminue*-t-elle quand N augmente ?

(c) « Train et test confondus » : est-ce une preuve que le modèle a appris la bonne loi ?

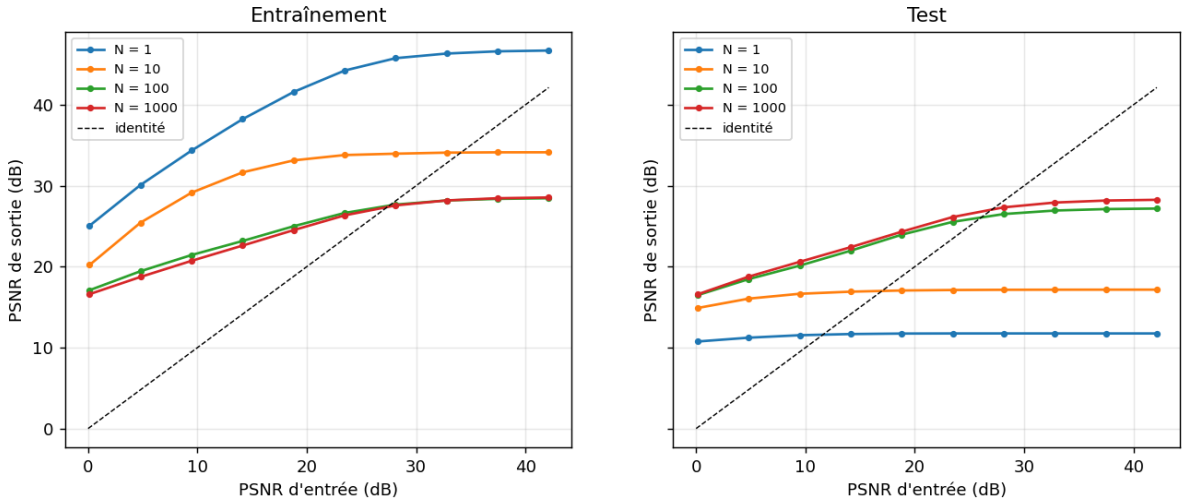
Ce qu'on ne peut pas refaire tel quel. Le notebook des auteurs `Demo_UNet_CelebA80x80_colab` recalcule la figure 1 avec leurs débruiteurs. Mais le jeu CelebA qu'il utilise ici a été reconstruit : même prétraitement, mais ordre des images et retrait des doublons différents.

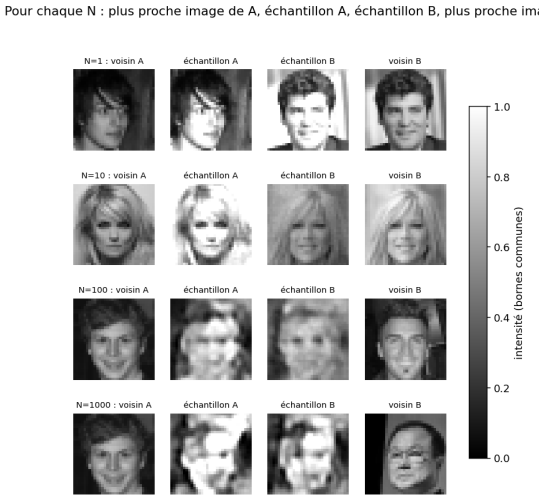
Q17 — Des courbes d'« entraînement » qui n'en sont pas

Exécutez la section « One shot denoising » de `Demo_UNet_CelebA80x80_colab`. Pour $N = 1$ et $N = 10$, les courbes d'« entraînement » ressemblent aux courbes de test de la figure 1. Pourquoi ? Que faudrait-il pour retrouver la vraie figure 1 ?

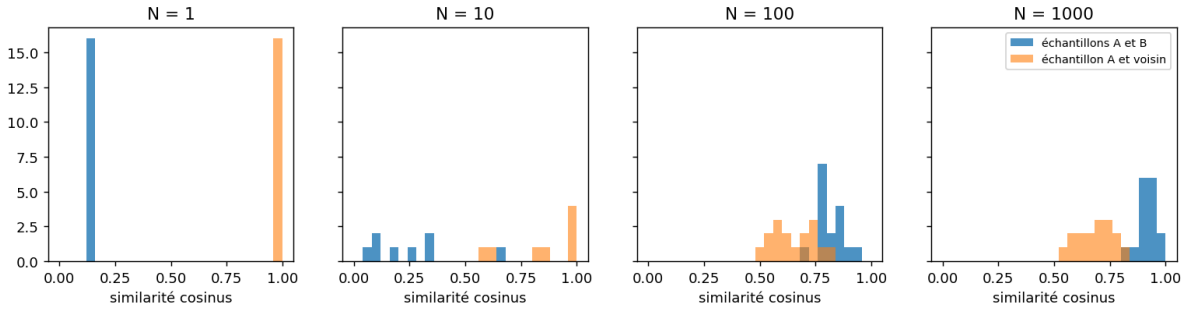
Entraîner soi-même

Le notebook `TP_entrainement_debruiteurs` entraîne de petits UNet sans biais (le code des auteurs, réduit), pour $N = 1, 10, 100, 1\,000$ (et $10\,000$ sur GPU), chaque fois deux réseaux sur des réservoirs disjoints A et B, avec la perte et le tirage du bruit des auteurs. Le profil CPU utilise des visages 32×32 et un réseau à 2 blocs et 16 canaux (114 496 paramètres) ; le profil GPU des visages 40×40 et 64 canaux. Il trace ensuite les courbes de la figure 1, puis compare les échantillons des modèles A et B partis du même bruit (figure 2).





Résultats du profil CPU (4 000 itérations par réseau, 28 minutes en tout). Pour chaque N : plus proche image du jeu A, échantillon du modèle A, échantillon du modèle B (même bruit initial, mêmes tirages), plus proche image du jeu B. Les paires dont un échantillon a divergé sont écartées (8 sur 16 pour $N = 10$, aucune ailleurs).



Q18 — Votre figure 1

(a) Retrouvez-vous les trois régimes de l'article ? Le notebook affiche l'écart moyen entre les PSNR d'entraînement et de test pour chaque N : à partir de quel N le réseau ne surapprend-il plus ?

(b) L'article trouve un seuil de l'ordre de 10^5 images pour des images 80×80 et un réseau de 7,6 millions de paramètres. Comment expliquez-vous un seuil bien plus bas ici ? (L'annexe C.4 de l'article étudie l'effet de la résolution.)

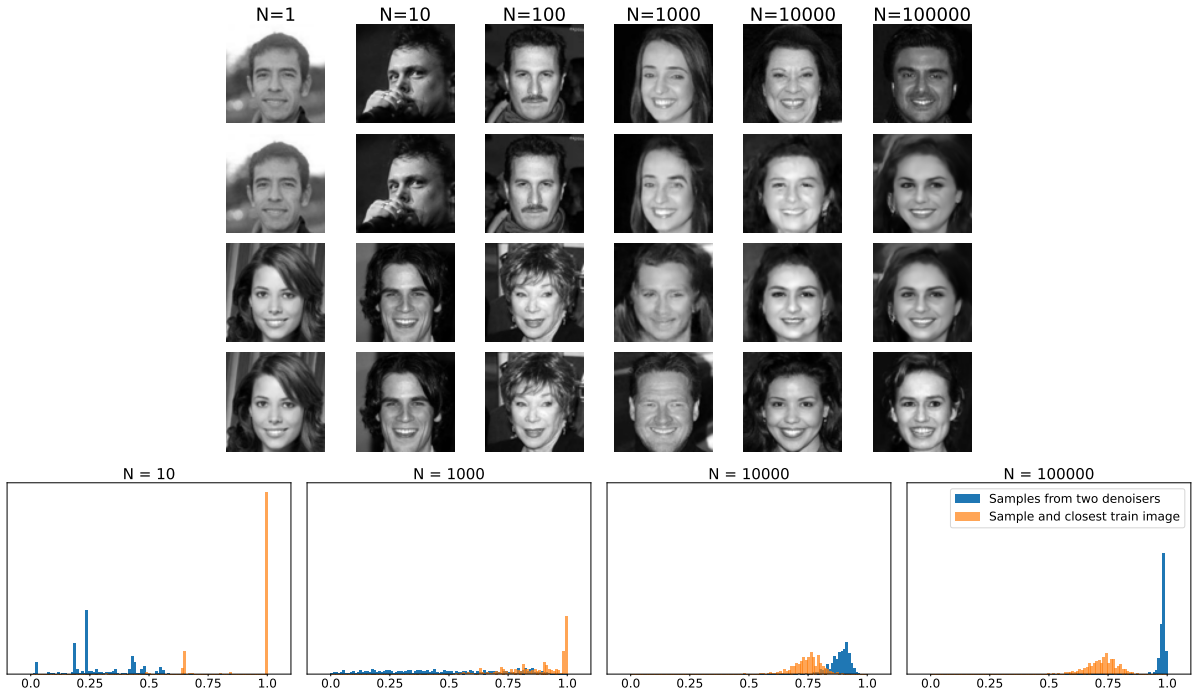


Figure 2 de l'article : de haut en bas, image de S_1 la plus proche, échantillon du modèle entraîné sur S_1, échantillon du modèle entraîné sur S_2 (même bruit initial), image de S_2 la plus proche, pour $N = 1$ à 10^5 ; en bas, histogrammes des similarités cosinus entre les échantillons des deux modèles (bleu) et entre un échantillon et l'image d'entraînement la plus proche (orange).

Q19 — Deux modèles, même bruit

(a) Décrivez la figure 2 de l'article : comment évoluent les deux histogrammes avec N ? Que signifie, pour $N = 10^5$, que deux réseaux entraînés sur des images *différentes* produisent presque la même image à partir du même bruit ?

(b) Pourquoi cette expérience est-elle une preuve de généralisation plus forte que la comparaison des erreurs d'entraînement et de test ?

(c) Comparez avec vos résultats (figures du notebook ci-dessus).

Retrouver les images mémorisées

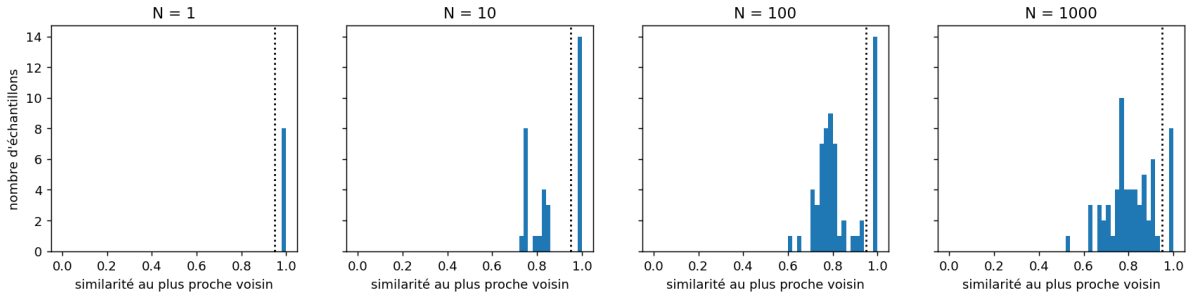
Si un modèle mémorise, ses échantillons sont des copies de ses images d'entraînement. Le notebook `TP_extraction_memorisation` génère des échantillons avec les modèles A des auteurs ($N = 1$ à $1\,000$; 8, 32, 64 et 64 échantillons sur CPU), puis cherche, pour chacun, l'image la plus semblable parmi les 198 650 images de CelebA reconstruit. C'est le principe des attaques par extraction de données d'entraînement (Carlini et al., 2023), qui ont montré que des modèles de diffusion publics recopient certaines de leurs images d'entraînement.





Q20 — Repérer la mémorisation sans les données

En regroupant les échantillons presque identiques (similarité cosinus supérieure à 0,95), le notebook trouve : 1 groupe pour 8 échantillons ($N = 1$), 13 pour 32 ($N = 10$), 37 pour 61 ($N = 100$, 3 échantillons divergents écartés), 60 pour 64 ($N = 1\,000$). Qu'en concluez-vous ? Un modèle qui mémorise N images peut-il produire plus de N groupes ? Regardez les échantillons de $N = 10$: que remarquez-vous ?



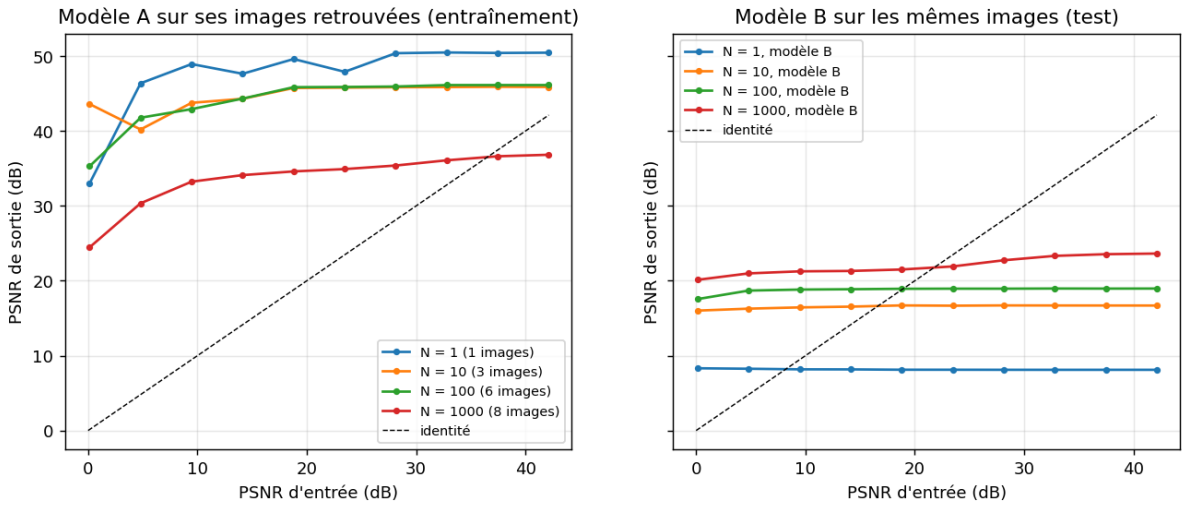


Q21 — Extraction

(a) Décrivez les histogrammes de la similarité au plus proche voisin dans CelebA. Pourquoi sont-ils bimodaux pour $N = 10$ et $N = 100$?

(b) Avec le seuil 0,95, le notebook retrouve 1 image pour $N = 1$, 3 pour $N = 10$, 6 pour $N = 100$ et 8 pour $N = 1\,000$. Pourquoi si peu pour $N = 10$, alors que l'histogramme compte une quinzaine de copies sur 32 échantillons ? Faites le lien avec la question 9.

(c) Les images retrouvées sont à des positions quelconques du fichier reconstruit (par exemple 3 466 pour $N = 1$), alors que les auteurs entraînent le modèle A sur les N premières images de leur fichier. Que confirme ce résultat ?



Q22 — La vraie figure 1, sur les images retrouvées

Les images retrouvées sont des images d'entraînement du modèle A et des images de test pour le modèle B (entraîné sur des données disjointes). Le notebook trace le PSNR de chacun sur ces images. Pour $N = 1$: de 50 à 33 dB pour A, 8 dB pour B ; pour $N = 10$: environ 46 dB pour A, 17 dB pour B ; pour $N = 1\,000$: de 37 à 25 dB pour A, de 24 à 20 dB pour B.

(a) Comparez à la figure 1 de l'article.

(b) Pour $N = 1\,000$, l'écart entre A et B reste grand, alors que l'article décrit un régime de transition. Pourquoi les images retrouvées ne sont-elles pas représentatives de l'ensemble du jeu d'entraînement ?

5 Le UNet sans biais : ce que l'architecture impose

Objectifs

- Lire l'architecture dans le code (`code/network.py`) et la comparer à celle des modèles de diffusion usuels.
- Comprendre ce que la suppression des biais permet (universalité, interprétabilité par le jacobien) et vérifier numériquement les propriétés sur lesquelles repose l'analyse de l'article.

L'architecture. Le réseau de l'article est un UNet à trois niveaux : un encodeur qui réduit trois fois la résolution ($80 \to 40 \to 20 \to 10$) en doublant le nombre de canaux ($64 \to 128 \to 256$), un bloc central (512 canaux), puis un décodeur symétrique qui remonte en résolution et reçoit, à chaque niveau, les caractéristiques de l'encodeur de même résolution (connexions de saut). Chaque bloc contient deux convolutions 3×3, chacune suivie d'une normalisation et d'une ReLU. Le champ récepteur couvre l'image entière (environ 90×90 pixels) ; le réseau compte 7,66 millions de paramètres. Il est *sans biais* : aucune convolution n'a de terme constant additif (`bias=False`), et la normalisation `BF_batchNorm` divise par un écart-type et multiplie par un gain, sans retrancher de moyenne ni ajouter de décalage :

$$\text{en apprentissage : } x \mapsto \gamma \frac{x}{\mathrm{sd}_{\text{lot}}(x)}, \qquad \text{en évaluation : } x \mapsto \gamma \frac{x}{\mathrm{sd}_{\text{moyen}}}.$$

Q23 — Un UNet simplifié

Les modèles de diffusion usuels (DDPM, Ho et al., 2020) utilisent aussi un UNet, mais avec : des biais dans toutes les couches ; une normalisation par groupes qui retranche la moyenne ; un plongement du temps t (ou de σ) ajouté dans chaque bloc ; des couches d'attention aux basses résolutions.

(a) Repérez dans `network.py` ce qui remplace ou supprime chacun de ces éléments.

(b) Le réseau de l'article ne reçoit pas σ. Comment peut-il néanmoins débruiter correctement à tous les niveaux de bruit ? Quel argument d'échelle le suggère ?

(c) Quel intérêt présente cette simplification pour l'article ? Qu'y perd-on ?

Q24 — Homogénéité et $f(y) = \nabla f(y)\, y$ (à la main, puis notebook)

(a) Montrez qu'une composition de convolutions sans biais, de ReLU, de moyennes 2×2, de concaténations et de `BF_batchNorm` *en mode évaluation* vérifie $f(\lambda y) = \lambda f(y)$ pour tout $\lambda > 0$. Pourquoi pas pour $\lambda < 0$? Pourquoi pas en mode apprentissage ?

(b) En dérivant $\lambda \mapsto f(\lambda y)$ en $\lambda = 1$, montrez que $f(y) = \nabla f(y)\, y$ (identité d'Euler). Interprétez : sur quelle région le réseau est-il exactement linéaire ?

(c) Le notebook `TP_jacobien_base_adaptative` (section 1) trouve : aucun paramètre nommé « bias » ; $\|f(\lambda y) - \lambda f(y)\|/\|\lambda f(y)\| = 0$ pour $\lambda = 0{,}5$ et 2, $5 \cdot 10^{-7}$ pour $\lambda = 10$, 0,87 pour $\lambda = -1$; $\|f(y) - \nabla f(y) y\|/\|f(y)\| = 0$; en mode apprentissage, l'écart d'homogénéité vaut $0{,}44$. Commentez.

(d) Un débruiteur qui mémorise une seule image x_1 voudrait renvoyer x_1 quelle que soit l'entrée (question 6). Un réseau sans biais le peut-il ? Que peut-il faire de plus proche ?

6 Le débruiteur comme rétrécissement dans une base adaptative

Objectifs

- Interpréter valeurs et vecteurs propres du jacobien d'un débruiteur ; relier le jacobien optimal à la covariance a posteriori et l'erreur à sa trace.
- Comparer la théorie (débruiteur optimal) et le réseau (asymétrie, trace, SURE).
- Voir la différence entre un réseau qui mémorise et un réseau qui généralise.

Une base adaptative. Si le jacobien $\nabla f(y)$ est symétrique positif, on peut le diagonaliser, $\nabla f(y) = \sum_k \lambda_k(y)\, e_k(y) e_k(y)^\top$, et, pour un réseau sans biais,

$$f(y) = \nabla f(y)\, y = \sum_k \lambda_k(y)\, \langle y, e_k(y)\rangle\, e_k(y). \tag{4}$$

C'est la même forme que le filtre de Wiener (Q11), mais la base $(e_k(y))$ et les facteurs $\lambda_k(y)$ dépendent de l'image observée : le débruiteur rétrécit les coefficients de y dans une base *adaptative*. Pour le débruiteur optimal, la relation de Miyasawa du second ordre (obtenue en dérivant une seconde fois, comme dans la question 1) donne

$$\nabla f^\star(y) = I + \sigma^2 \nabla^2 \log p_\sigma(y) = \frac{\mathrm{Cov}[x \mid y]}{\sigma^2}, \qquad \mathrm{MSE}(f^\star, \sigma^2) = \mathbb{E}_y\big[\,\mathrm{tr}\,\mathrm{Cov}[x \mid y]\big] = \sigma^2\, \mathbb{E}_y\big[\,\mathrm{tr}\,\nabla f^\star(y)\big]. \tag{5}$$

Q25 — Jacobien, covariance et erreur (à la main)

(a) Pourquoi le jacobien du débruiteur optimal est-il symétrique et positif ? Que représentent ses vecteurs propres et ses valeurs propres pour la loi a posteriori ?

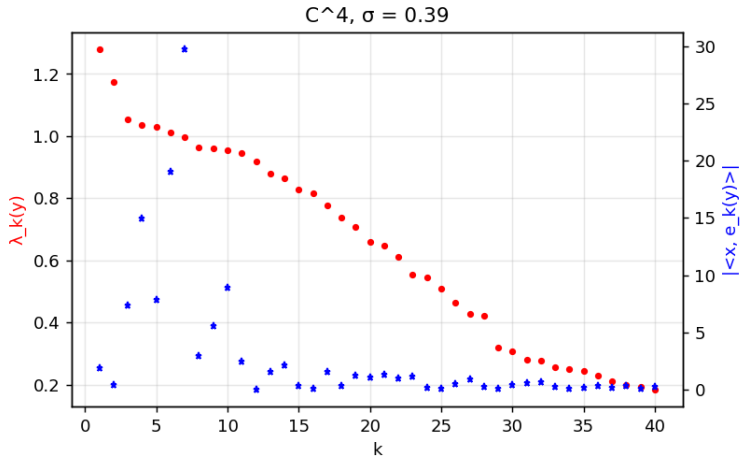
(b) *Estimateur de Stein (SURE).* Pour *tout* débruiteur f (dérivable), montrez que

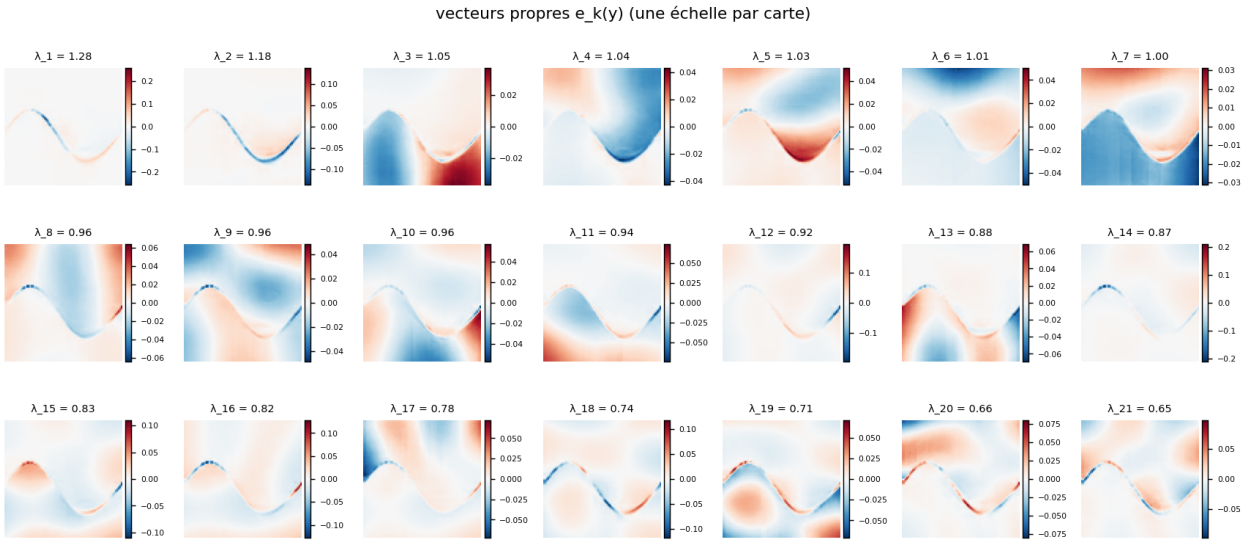
$$\mathrm{MSE}(f, \sigma^2) = \mathbb{E}_y\big[\|y - f(y)\|^2 + 2\sigma^2\, \mathrm{tr}\,\nabla f(y) - \sigma^2 d\big],$$

en développant $\|x - f(y)\|^2 = \|(y - f(y)) - (y - x)\|^2$ et en admettant le lemme de Stein : $\mathbb{E}[\langle z, g(x+z)\rangle] = \sigma^2\, \mathbb{E}[\mathrm{tr}\,\nabla g(x+z)]$ pour $z \sim \mathcal{N}(0, \sigma^2 I)$.

(c) Interprétez SURE comme un compromis : que coûte une grande trace ? Pourquoi dit-on que le débruiteur se comporte comme une projection (douce) sur un sous-espace de dimension $\mathrm{tr}\,\nabla f$?

Mesures sur le réseau $\mathbf{C}^4$. Le notebook `TP_jacobien_base_adaptative` (section 2) analyse le réseau des auteurs entraîné sur 10^5 images $\mathbf{C}^4$, pour une image de test bruitée à $\sigma = 100/255$: asymétrie, valeurs propres $\lambda_k(y)$ de la partie symétrique, coefficients $|\langle x, e_k(y)\rangle|$ de l'image *propre*, vecteurs propres.





Q26 — Une base adaptée à l'image

(a) Décrivez les vecteurs propres. En quoi sont-ils « adaptés à la géométrie » de l'image ? Comment leur fréquence évolue-t-elle avec k ?

(b) Les 40 premiers vecteurs propres portent 99,7 % de l'énergie de l'image propre. Que signifie ce résultat en termes de parcimonie ? Pourquoi est-il remarquable que la base ait été calculée à partir de y et non de x ?

(c) Les premières valeurs propres dépassent 1 (1,28 ; 1,18 ; 1,05…) et l'asymétrie relative $\|Jv - J^\top v\|/\|Jv\|$ vaut 0,49. Ces deux constats sont-ils compatibles avec le débruiteur optimal ? avec ce que suppose l'article ? Comment le code des auteurs (`torch.svd(I - J)` dans leurs notebooks) traite-t-il cette question ?

Trace, dimension effective et erreur. Le notebook (section 3) estime la trace du jacobien sans le former (méthode de Hutchinson : $\operatorname{tr} J = \mathbb{E}[v^\top Jv]$ pour v à composantes ± 1) sur 20 images, à six niveaux de bruit, et compare l'erreur réelle, l'estimation SURE et $\sigma^2 \operatorname{tr} \nabla f$.

σ	tr ∇f	erreur $\|x - f(y)\|^2$	SURE	σ^2 tr ∇f
0,020	530	0,14	0,14	0,20
0,039	417	0,26	0,27	0,64
0,078	268	0,44	0,50	1,65
0,196	233	1,90	2,23	8,94
0,392	219	5,60	7,40	33,7
0,784	212	17,5	25,7	130,5

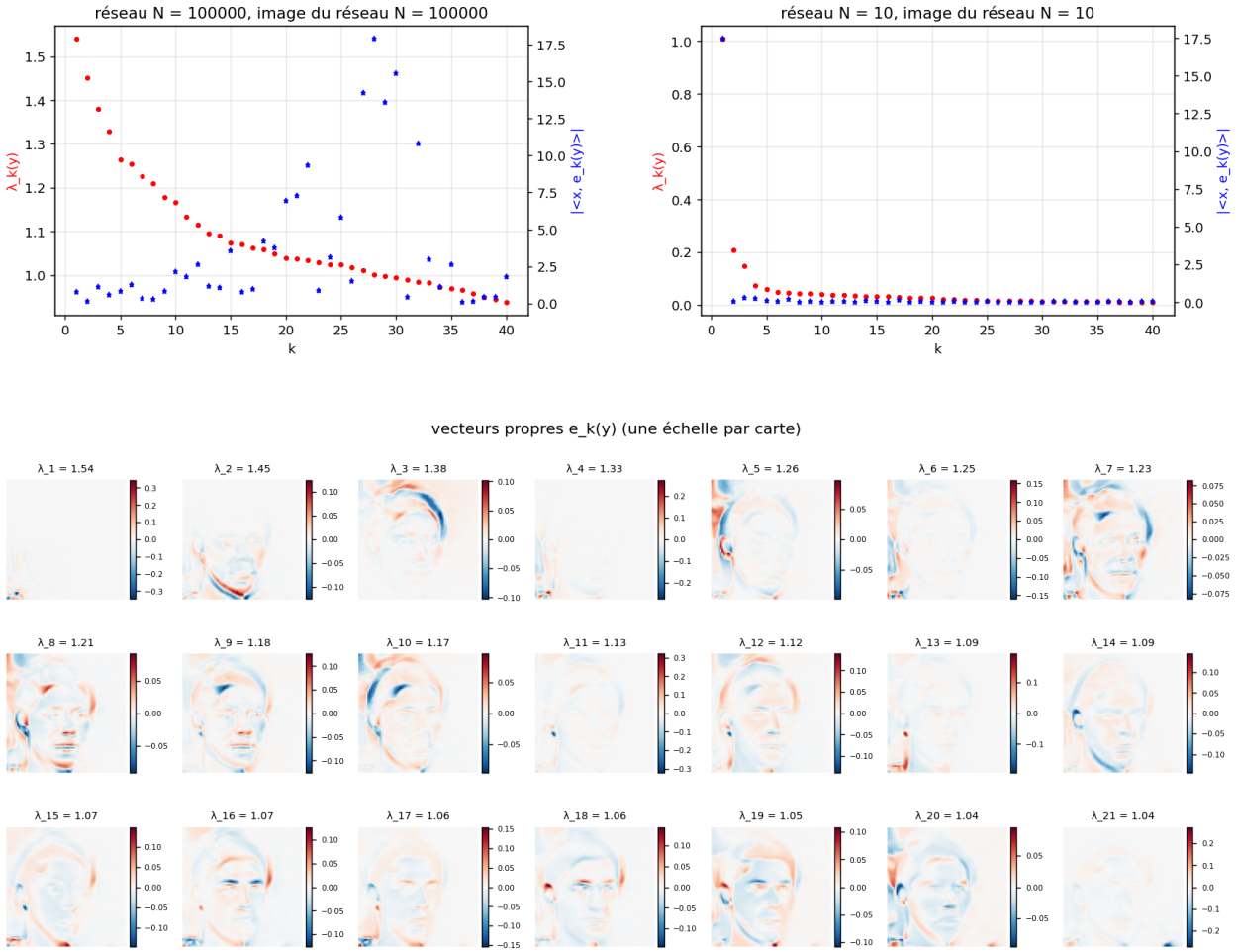
Q27 — Que dit la trace ?

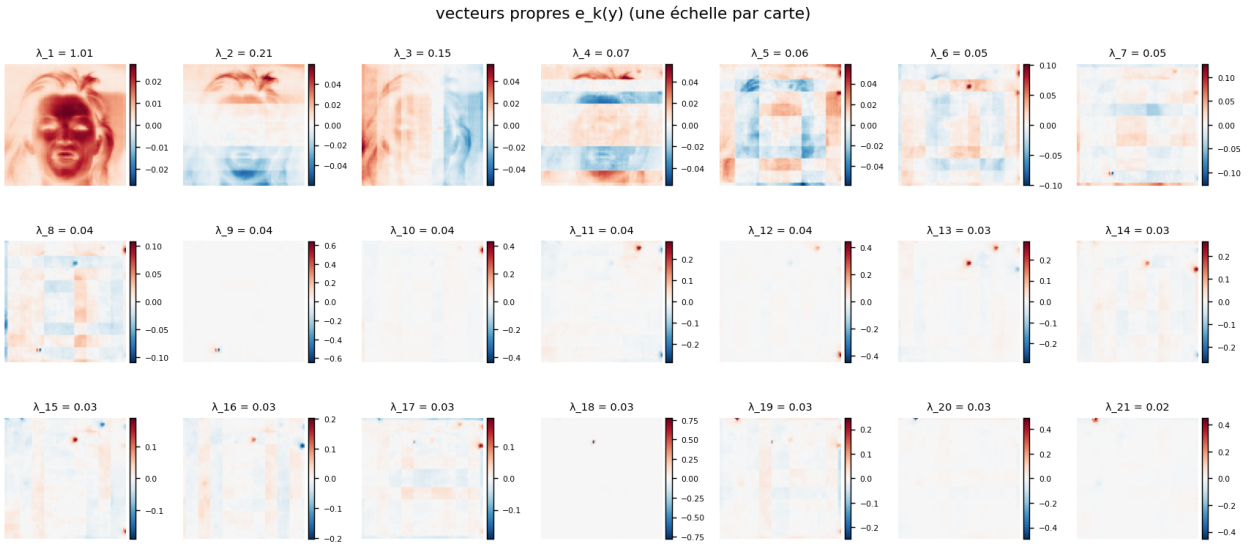
(a) Comment la dimension effective tr ∇f évolue-t-elle avec le bruit ? Est-ce cohérent avec la question 13 ?

(b) SURE suit-il l'erreur réelle ? Pourquoi l'écart grandit-il avec σ (pensez à la taille des termes qui se compensent dans SURE, et au nombre d'images) ?

(c) D'après (5), que vaudrait σ^2 tr ∇f pour le débruiteur optimal ? Que conclure du tableau ? Est-ce contradictoire avec une pente « optimale » ?

Mémoriser ou généraliser, vu par le jacobien. Section 4 du notebook : les débruiteurs CelebA des auteurs entraînés sur $N = 10^5$ et $N = 10$ images, analysés à $\sigma = 40/255$ sur une image produite par chacun d'eux (faute des images d'entraînement, on utilise les échantillons sauvegardés par les auteurs ; pour $N = 10$, ce sont des copies d'images d'entraînement). La figure 3 de l'article montre la même analyse pour $N = 10^5$ sur une image de test.





Q28 — Deux réseaux, deux jacobiens

(a) Comparez les valeurs propres des deux réseaux (à gauche $N = 10^5$: 1,54 ; 1,45 ; 1,38…; à droite $N = 10$: 1,01 ; 0,21 ; 0,15 ; 0,07…) et leurs premiers vecteurs propres (en haut $N = 10^5$, en bas $N = 10$). Reliez le cas $N = 10$ à la question 24(d).

(b) Le réseau $N = 10$ appliqué à un visage nouveau (le notebook le fait aussi) a une asymétrie de 1,42 et des valeurs propres jusqu'à 2,6. Qu'en penser ?

(c) D'après les parties 2 et 5, pourquoi un réseau qui généralise doit-il avoir un jacobien de rang élevé et adapté à chaque image, et un réseau qui mémorise un jacobien de rang très faible ?

7 Bases harmoniques adaptées à la géométrie

Objectifs

- Comprendre pourquoi les images $\mathbf{C}^\alpha$ permettent de tester l'optimalité du réseau.
- Vérifier la pente $\alpha/(\alpha+1)$ et discuter des limites du biais inductif.

Les images $\mathbf{C}^\alpha$. Une image géométrique $\mathbf{C}^\alpha$ est formée de deux régions séparées par un contour ; l'intensité est α fois régulière dans chaque région (au sens de Hölder : approchable localement par un polynôme de degré $\lfloor \alpha \rfloor$ avec une erreur en $|u|^\alpha$), et le contour est une courbe α fois régulière. Les images sont produites par `make_C_alpha_images` (`code/synthetic_data_generators.py`) : un contour obtenu en intégrant un bruit 1D (filtre $|\omega|^{-\alpha}$ dans le domaine de Fourier), deux fonds obtenus de même en 2D, et un masque qui les combine. Pour cette classe, on connaît la meilleure représentation : les *bandelettes* (Le Pennec, Mallat, Peyré), des oscillations de fréquences croissantes alignées sur la direction locale du contour, dans lesquelles le k-ième coefficient décroît comme $k^{-(\alpha+1)}$. La meilleure pente atteignable est donc $\alpha/(\alpha+1)$ (Q13), et un estimateur qui choisit la meilleure base de bandelettes d'après y l'atteint (Dossal, Mallat, Peyré, 2011).

Q29 — Le réseau est-il optimal sur les images $\mathbf{C}^\alpha$? (notebook des auteurs)

Exécutez `Demo_UNet_Calpha80x80_colab` (section « One shot denoising ») : il trace les courbes PSNR des réseaux entraînés sur 10^5 images $\mathbf{C}^\alpha$ pour $\alpha = 1, 2, 3, 5$ et compare la pente mesurée à $\alpha/(\alpha+1)$ (entre parenthèses). Comparez avec la figure 4 de l'article.

(a) Les pentes mesurées sont-elles proches des pentes théoriques ? Pour quelles valeurs de α l'écart est-il le plus grand ? Le notebook ajuste la pente sur *tous* les niveaux de bruit : est-ce conforme à ce que dit la théorie ?

(b) Dans la partie 3, le filtre de Wiener obtenait une pente de 0,59 sur les images $\mathbf{C}^4$, le réseau 0,80. Que cela dit-il de la base qu'utilise le réseau ?

(c) Pourquoi ce résultat est-il un argument plus fort que la qualité visuelle des visages générés ?

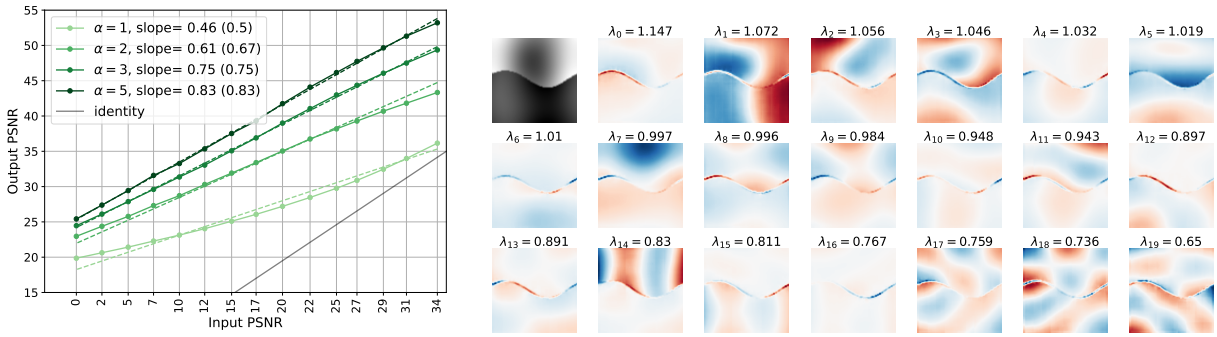


Figure 4 de l'article [1] : courbes PSNR pour α variable (pentes mesurées et, entre parenthèses, théoriques) ; une image $\mathbf{C}^4$ et les premiers vecteurs propres du jacobien.

Q30 — Pourquoi pas la base parfaite ? (à la main)

L'article remarque que la meilleure base « oracle » pour une image x commence par $e_1 = x/\|x\|$: un seul coefficient non nul. Pourquoi un débruiteur ne peut-il pas utiliser $e_1(y) = y/\|y\|$? Pourquoi faut-il choisir la base dans une famille (un « dictionnaire ») assez riche mais assez petite ? Quel rapport avec la partie 2 ?

Les limites du biais. Si le réseau est biaisé vers les GAHB, il doit être sous-optimal quand la bonne base n'en est pas une. L'article l'étudie sur deux exemples :

- des images de disques (position, rayon, intensités du disque et du fond variables) : elles forment une variété de dimension 5, et pour un petit bruit le débruiteur optimal projette sur l'espace tangent (5 vecteurs, pente 1) ; le réseau retrouve ces 5 vecteurs, mais en conserve d'autres, de type GAHB (oscillations le long du bord et dans le fond), et sa pente est inférieure à 1 ;
- des visages dont les pixels ont été mélangés par une même permutation : l'optimum est aussi bon que pour les vrais visages (il suffit de démélanger), mais le réseau, dont les convolutions supposent que les pixels voisins sont liés, débruite nettement moins bien.

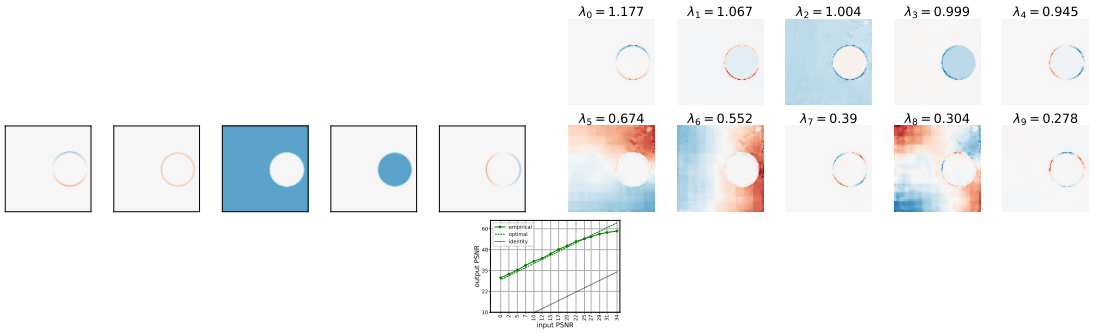


Figure 5 de l'article : disques. À gauche, une base de l'espace tangent (optimale pour un petit bruit) ; au centre, les premiers vecteurs propres du réseau (5 tangents, puis des GAHB) ; à droite, pente PSNR inférieure à 1.

Q31 — Biais inductif : une qualité ou un défaut ? (à la main)

(a) Pourquoi une permutation des pixels ne change-t-elle pas la difficulté du problème pour l'optimum, mais la change pour le réseau ? Quelles opérations de l'architecture supposent la localité ?

(b) Distinguez la *variance* du modèle et son *biais* au sens de l'article. Lequel la convergence de deux réseaux entraînés sur des données disjointes (partie 4) mesure-t-elle ? Lequel les expériences de cette partie mesurent-elles ?

Bilan

- Le minimum exact de l'erreur de débruitage sur un jeu fini est un débruiteur qui recopie ses images d'entraînement ; en grande dimension, c'est vrai à tous les niveaux de bruit utiles (partie 2).
- Un débruiteur linéaire (optimal pour une loi gaussienne) travaille dans une base fixe ; sur des images à contours, il est loin du compte (partie 3).
- Les réseaux de l'article mémorisent quand N est petit et généralisent quand N est grand : deux réseaux entraînés sur des images différentes apprennent alors presque la même fonction (partie 4).
- Le UNet sans biais est, en chaque point, un opérateur linéaire ; son jacobien révèle un rétrécissement dans une base adaptée à l'image, faite d'oscillations qui suivent ses contours (parties 5 et 6).
- Pour les images $\mathbf{C}^\alpha$, cette base permet d'atteindre la meilleure performance possible ; pour d'autres données, le même biais rend le réseau sous-optimal (partie 7).

Références

[1] Z. Kadkhodaie, F. Guth, E. P. Simoncelli, S. Mallat, *Generalization in diffusion models arises from geometry-adaptive harmonic representations*, ICLR 2024 (arXiv:2310.02557).

[2] Z. Kadkhodaie, E. P. Simoncelli, *Stochastic solutions for linear inverse problems using the prior implicit in a denoiser*, NeurIPS 2021.

[3] S. Mohan, Z. Kadkhodaie, E. P. Simoncelli, C. Fernandez-Granda, *Robust and interpretable blind image denoising via bias-free convolutional neural networks*, ICLR 2020.

[4] K. Miyasawa, *An empirical Bayes estimator of the mean of a normal population*, Bull. Inst. Internat. Statist., 1961 ; M. Raphan, E. P. Simoncelli, *Least squares estimation without priors or supervision*, Neural Computation, 2011.

[5] Y. Song, C. Durkan, I. Murray, S. Ermon, *Maximum likelihood training of score-based diffusion models*, NeurIPS 2021.

[6] J. Ho, A. Jain, P. Abbeel, *Denoising diffusion probabilistic models*, NeurIPS 2020.

[7] N. Carlini et al., *Extracting training data from diffusion models*, USENIX Security 2023 ; G. Somepalli et al., *Diffusion art or digital forgery?*, CVPR 2023.

[8] D. Donoho, I. Johnstone, *Ideal spatial adaptation by wavelet shrinkage*, Biometrika, 1994.

[9] G. Peyré, S. Mallat, *Orthogonal bandelet bases for geometric images approximation*, Comm. Pure Appl. Math., 2008 ; C. Dossal, S. Mallat, E. Le Pennec, *Bandlet image estimation with model selection*, Signal Processing, 2011.

[10] S. Mallat, *A wavelet tour of signal processing: the sparse way*, Academic Press, 2009.